

Uppsala University
Department of Information Technology

Assignment 2

Data Stream Processing

Course: Data Engineering II

Arnab Kumar Ghosh

April 22, 2026

Contents

1 Task 1: Setting up Apache Pulsar	2
1.1 Setting up the VM	2
1.2 Setting up Pulsar in a container in a VM	2
1.3 Running the Program	2
2 Task 2: Producing and Consuming Messages	3
2.1 A: Running Apache Pulsar, consumer and producer on a single machine	3
2.1.1 Installing the Python Client Library	3
2.1.2 The Consumer	3
2.1.3 The Producer	3
2.1.4 Running the Programs	3
2.2 B: Running Apache Pulsar, consumer and producer on multiple machines	5
2.2.1 Infrastructure	5
2.2.2 Which IP Address to Use?	5
2.2.3 Starting Pulsar on VM A	5
2.2.4 The Consumer on VM B	5
2.2.5 Running the Programs	5
3 Task 3: Conceptual Questions	6
4 Task 4 : Splitting and Merging with Apache Pulsar	9
4.1 Issue with the Current Implementation	9
4.2 Redesigned Architecture	9
4.3 Implementation	9
4.4 Running the Programs	10
A Code Files	11
A.1 Task 2.A: Consumer	11
A.2 Task 2.A: Producer	11
A.3 Task 2.B: Consumer	12
A.4 Task 4: Producer	12
A.5 Task 4: Worker	13
A.6 Task 4: Merger	14
B VM Setup Script	15

1 Task 1: Setting up Apache Pulsar

Apache Pulsar is an open-source distributed messaging and streaming platform. Before we can produce or consume messages, we need a running Pulsar service. There are two ways to do this: installing Pulsar directly on the VM, or running it inside a Docker container. In this task, I used the Docker method because it is faster to set up and keeps the VM clean.

1.1 Setting up the VM

Before starting Pulsar, the VM needs Java and Docker. I wrote a setup script `setup_vm.sh` to install all prerequisites automatically.

The script does the following steps:

- Updates all system packages with `apt update` and `apt upgrade`.
- Installs the Java Runtime Environment, which Pulsar requires to run.
- Installs Docker CE using Docker's official repository and starts the Docker service.
- Installs `python3-pip` and the `pulsar-client==2.9.4` Python library.

1.2 Setting up Pulsar in a container in a VM

Once the VM was ready, I started Apache Pulsar in standalone mode using the official Docker image. Standalone mode runs the broker, BookKeeper, and ZooKeeper all in one process, which is enough for development and testing.

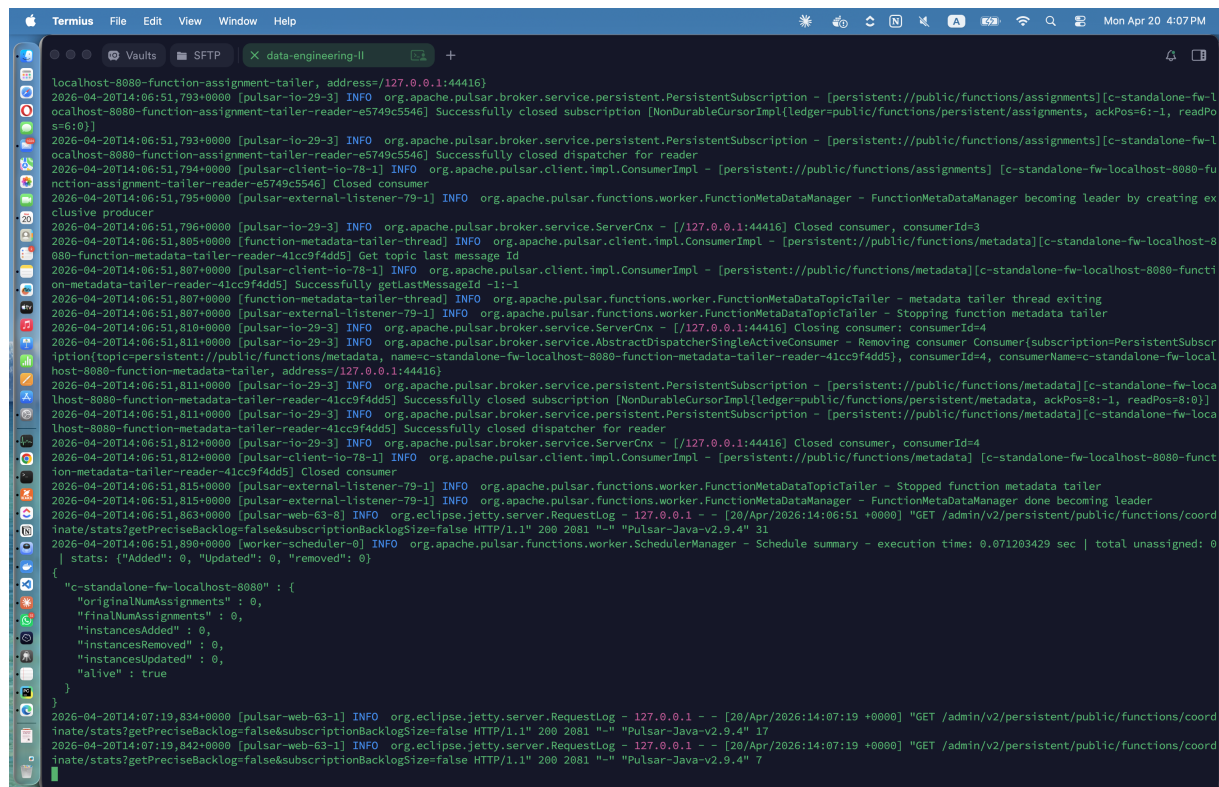
Listing 1: Starting Apache Pulsar in a Docker container

```
docker run -it \  
-p 6650:6650 \  
-p 8080:8080 \  
--mount source=pulsardata,target=/pulsar/data \  
--mount source=pulsarconf,target=/pulsar/conf \  
apachepulsar/pulsar:2.9.4 bin/pulsar standalone
```

- `-p 6650:6650` – Exposes the Pulsar broker port so clients can connect to it.
- `-p 8080:8080` – Exposes the Pulsar admin REST API port.
- `-mount source=pulsardata` – Mounts a named Docker volume for persistent message data. Data survives even if the container is restarted.
- `-mount source=pulsarconf` – Mounts a separate volume for Pulsar configuration files.
- `bin/pulsar standalone` – Starts Pulsar in standalone mode.

1.3 Running the Program

When Pulsar starts successfully, the logs show a JSON status block with `"alive": true` and regular HTTP request lines from the admin API. This confirms that the broker is running and healthy.



```

localhost-8880-function-assignment-tailer, address=127.0.0.1:44416
2026-04-20T14:06:51,793+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.persistent.PersistentSubscription - [persistent://public/functions/assignments][c-standalone-fw-l
ocalhost-8880-function-assignment-tailer-reader-e5749c5546] Successfully closed subscription [NonDurableCursorImpl{ledger=public/functions/persistent/assignments, ackPos=6:-1, readPos=6:0}]
2026-04-20T14:06:51,793+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.persistent.PersistentSubscription - [persistent://public/functions/assignments][c-standalone-fw-l
ocalhost-8880-function-assignment-tailer-reader-e5749c5546] Successfully closed dispatcher for reader
2026-04-20T14:06:51,794+0000 [pulsar-client-io-78-1] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/functions/assignments] [c-standalone-fw-localhost-8880-fu
nction-assignment-tailer-reader-e5749c5546] Closed consumer
2026-04-20T14:06:51,795+0000 [pulsar-external-listener-79-1] INFO org.apache.pulsar.functions.worker.FunctionMetadataManager - FunctionMetadataManager becoming leader by creating ex
clusive producer
2026-04-20T14:06:51,796+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.ServerCnx - [127.0.0.1:44416] Closed consumer, consumerId=3
2026-04-20T14:06:51,805+0000 [function-metadata-tailer-thread] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/functions/metadata][c-standalone-fw-localhost-8
880-function-metadata-tailer-reader-41cc9f4dd5] Get topic last message Id
2026-04-20T14:06:51,807+0000 [pulsar-client-io-78-1] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/functions/metadata][c-standalone-fw-localhost-8880-functi
on-metadata-tailer-reader-41cc9f4dd5] Successfully get last message Id -1-1
2026-04-20T14:06:51,807+0000 [function-metadata-tailer-thread] INFO org.apache.pulsar.functions.worker.FunctionMetadataTopicTailer - metadata tailer thread exiting
2026-04-20T14:06:51,807+0000 [pulsar-external-listener-79-1] INFO org.apache.pulsar.functions.worker.FunctionMetadataTopicTailer - Stopping function metadata tailer
2026-04-20T14:06:51,810+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.ServerCnx - [127.0.0.1:44416] Closing consumer: consumerId=4
2026-04-20T14:06:51,811+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.AbstractDispatcherSingleActiveConsumer - Removing consumer Consumer{subscription=PersistentSubscr
ption(topic=persistent://public/functions/metadata, name=c-standalone-fw-localhost-8880-function-metadata-tailer-reader-41cc9f4dd5), consumerId=4, consumerName=c-standalone-fw-local
host-8880-function-metadata-tailer, address=127.0.0.1:44416}
2026-04-20T14:06:51,811+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.persistent.PersistentSubscription - [persistent://public/functions/metadata][c-standalone-fw-loc
alhost-8880-function-metadata-tailer-reader-41cc9f4dd5] Successfully closed subscription [NonDurableCursorImpl{ledger=public/functions/persistent/metadata, ackPos=8:-1, readPos=8:0}]
2026-04-20T14:06:51,811+0000 [pulsar-io-29-3] INFO org.apache.pulsar.broker.service.persistent.PersistentSubscription - [persistent://public/functions/metadata][c-standalone-fw-loc
alhost-8880-function-metadata-tailer-reader-41cc9f4dd5] Successfully closed dispatcher for reader
2026-04-20T14:06:51,812+0000 [pulsar-client-io-78-1] INFO org.apache.pulsar.broker.service.ServerCnx - [127.0.0.1:44416] Closed consumer, consumerId=4
2026-04-20T14:06:51,812+0000 [pulsar-client-io-78-1] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/functions/metadata] [c-standalone-fw-localhost-8880-functi
on-metadata-tailer-reader-41cc9f4dd5] Closed consumer
2026-04-20T14:06:51,815+0000 [pulsar-external-listener-79-1] INFO org.apache.pulsar.functions.worker.FunctionMetadataTopicTailer - Stopped function metadata tailer
2026-04-20T14:06:51,815+0000 [pulsar-external-listener-79-1] INFO org.apache.pulsar.functions.worker.FunctionMetadataManager - FunctionMetadataManager done becoming leader
2026-04-20T14:06:51,863+0000 [pulsar-web-63-8] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [20/Apr/2026:14:06:51 +0000] "GET /admin/v2/persistent/public/functions/coord
inate/stats?getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 31
2026-04-20T14:06:51,890+0000 [worker-scheduler-0] INFO org.apache.pulsar.functions.worker.SchedulerManager - Schedule summary - execution time: 0.071203429 sec | total unassigned: 0
{
  "stats": {
    "Added": 0,
    "Updated": 0,
    "Removed": 0
  }
}
{
  "c-standalone-fw-localhost-8880": {
    "originalNumAssignments": 0,
    "finalNumAssignments": 0,
    "instancesAdded": 0,
    "instancesRemoved": 0,
    "instancesUpdated": 0,
    "alive": true
  }
}
2026-04-20T14:07:19,834+0000 [pulsar-web-63-1] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [20/Apr/2026:14:07:19 +0000] "GET /admin/v2/persistent/public/functions/coord
inate/stats?getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 17
2026-04-20T14:07:19,842+0000 [pulsar-web-63-1] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [20/Apr/2026:14:07:19 +0000] "GET /admin/v2/persistent/public/functions/coord
inate/stats?getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 7

```

Figure 1: Apache Pulsar running successfully in standalone mode inside a Docker container.

2 Task 2: Producing and Consuming Messages

2.1 A: Running Apache Pulsar, consumer and producer on a single machine

With Pulsar running, we can now write a producer and a consumer. In this part, both programs run on the same VM that hosts the Pulsar broker.

2.1.1 Installing the Python Client Library

Before running any Python code, the Pulsar client library must be installed. The setup script already handles this, but the command is:

```
pip install pulsar-client==2.9.4
```

2.1.2 The Consumer

The consumer subscribes to a topic and waits for messages. It acknowledges each message it receives so Pulsar knows the message was processed. The implementation of `consumer-task-2-A.py` is shown in Appendix A.

2.1.3 The Producer

The producer connects to the same topic and sends one message. The implementation of `producer-task-2.py` is shown in Appendix A.

2.1.4 Running the Programs

I opened three separate SSH sessions to the VM. Pulsar was already running in the first session. I started the consumer in the second session and left it waiting. I then ran the producer in

the third session.

Listing 2: Running the consumer and producer

```
# Terminal 2: start the consumer and keep it waiting
python3 consumer-task-2-A.py
```

```
# Terminal 3: run the producer to send a message
python3 producer-task-2.py
```

As soon as the producer sent the message, the consumer printed:

```
Received message : 'b'Welcome to Data Engineering Course!'
```

This confirms that the message was successfully delivered through the Pulsar broker.

```
data-engineering-II (2) ssh, ubuntu
ubuntu@arnab-de2-a2-t1:~$ python3 producer.py
2026-04-20 14:26:58.253 INFO [14035460998464] ExecutorService:41 | Run io_service in a single thread
2026-04-20 14:26:58.253 INFO [140354620985792] ClientConnection:189 | [none] -> pulsar://localhost:6650] Create ClientConnection, timeout=10000
2026-04-20 14:26:58.254 INFO [140354620985792] ConnectionPool:96 | Created connection for pulsar://localhost:6650
2026-04-20 14:26:58.255 INFO [14035460998464] ClientConnection:375 | [127.0.0.1:41586 -> 127.0.0.1:6650] Connected to broker
2026-04-20 14:26:58.263 INFO [14035460998464] HandlerBase:61 | [persistent://public/default/DEtopic, ] Getting connection from pool
2026-04-20 14:26:58.270 INFO [14035460998464] ProducerImpl:173 | [persistent://public/default/DEtopic, ] Created producer on broker [127.0.0.1:41586 -> 127.0.0.1:6650]
2026-04-20 14:26:58.288 INFO [140354620985792] ClientImpl:500 | Closing Pulsar client with 1 producers and 0 consumers
2026-04-20 14:26:58.289 INFO [140354620985792] ProducerImpl:590 | [persistent://public/default/DEtopic, standalone-0-0] Closing producer for topic persistent://public/default/DEtopic
2026-04-20 14:26:58.296 INFO [14035460998464] ProducerImpl:630 | [persistent://public/default/DEtopic, standalone-0-0] Closed producer
2026-04-20 14:26:58.296 INFO [140354584213056] ClientConnection:1561 | [127.0.0.1:41586 -> 127.0.0.1:6650] Connection closed
2026-04-20 14:26:58.297 INFO [140354584213056] ClientConnection:263 | [127.0.0.1:41586 -> 127.0.0.1:6650] Destroyed connection
2026-04-20 14:26:58.299 INFO [14035460998464] ExecutorService:47 | Event loop of ExecutorService exits successfully
2026-04-20 14:26:58.299 INFO [140354620985792] ProducerImpl:559 | Producer - [persistent://public/default/DEtopic, standalone-0-0] , [batching = off]
ubuntu@arnab-de2-a2-t1:~$

data-engineering-II (1) ssh, ubuntu
ubuntu@arnab-de2-a2-t1:~$ python3 consumer.py
2026-04-20 14:26:45.887 INFO [139644830257600] Client:88 | Subscribing on Topic :DEtopic
2026-04-20 14:26:45.887 INFO [139644810270272] ExecutorService:41 | Run io_service in a single thread
2026-04-20 14:26:45.887 INFO [139644830257600] ClientConnection:189 | [none] -> pulsar://localhost:6650] Create ClientConnection, timeout=10000
2026-04-20 14:26:45.887 INFO [139644830257600] ConnectionPool:96 | Created connection for pulsar://localhost:6650
2026-04-20 14:26:45.889 INFO [139644810270272] ClientConnection:375 | [127.0.0.1:51322 -> 127.0.0.1:6650] Connected to broker
2026-04-20 14:26:45.897 INFO [139644810270272] HandlerBase:61 | [persistent://public/default/DEtopic, DE-sub, 0] Getting connection from pool
2026-04-20 14:26:45.897 INFO [139644793484864] ExecutorService:41 | Run io_service in a single thread
2026-04-20 14:26:45.902 INFO [139644810270272] ConsumerImpl:218 | [persistent://public/default/DEtopic, DE-sub, 0] Created consumer on broker [127.0.0.1:51322 -> 127.0.0.1:6650]
Received message : 'b'Welcome
to Data Engineering Course!'
2026-04-20 14:26:58.304 INFO [139644830257600] ClientImpl:500 | Closing Pulsar client with 0 producers and 1 consumers
2026-04-20 14:26:58.312 INFO [139644810270272] ConsumerImpl:885 | [persistent://public/default/DEtopic, DE-sub, 0] Closing consumer for topic persistent://public/default/DEtopic
2026-04-20 14:26:58.312 INFO [139644810270272] ConsumerImpl:935 | [persistent://public/default/DEtopic, DE-sub, 0] Closed consumer 0
2026-04-20 14:26:58.312 INFO [139644785092160] ClientConnection:1561 | [127.0.0.1:51322 -> 127.0.0.1:6650] Connection closed
2026-04-20 14:26:58.312 INFO [139644785092160] ClientConnection:263 | [127.0.0.1:51322 -> 127.0.0.1:6650] Destroyed connection
2026-04-20 14:26:58.313 INFO [139644810270272] ExecutorService:47 | Event loop of ExecutorService exits successfully
2026-04-20 14:26:58.313 INFO [139644793484864] ExecutorService:47 | Event loop of ExecutorService exits successfully
ubuntu@arnab-de2-a2-t1:~$ tmux ls
error connecting to /tmp/tmux-1000/default (No such file or directory)
ubuntu@arnab-de2-a2-t1:~$
```

Figure 2: Consumer receiving a message sent by the producer on the same machine.

2.2 B: Running Apache Pulsar, consumer and producer on multiple machines

In this part, the setup is distributed across two separate virtual machines on the Swedish Science Cloud (NAISS). Apache Pulsar and the producer run on **VM A**, while the consumer runs on **VM B**.

2.2.1 Infrastructure

- **VM A** (arnab-de2-a2-t1): private IP 192.168.2.198: runs the Pulsar broker in Docker and the producer.
- **VM B** (arnab-de2-a2-t2): private IP 192.168.2.166: runs the consumer only. No Docker is required on this machine.

2.2.2 Which IP Address to Use?

The private IP address should be used. Both VMs are on the same Swedish Science Cloud (NAISS) internal network, so VM B can reach VM A directly using the private IP (192.168.2.198). The traffic stays inside the data centre and does not leave the internal network. The public floating IP (130.*.*.*) routes traffic externally and would require opening port 6650 in the security group, which adds unnecessary exposure. Therefore, I used the private IP address in the consumer's broker URL.

2.2.3 Starting Pulsar on VM A

I used the same Docker command as in Task 1 without any changes:

Listing 3: Starting Pulsar on VM A – same command as Task 1

```
docker run -it \  
-p 6650:6650 \  
-p 8080:8080 \  
--mount source=pulsardata,target=/pulsar/data \  
--mount source=pulsarconf,target=/pulsar/conf \  
apachepulsar/pulsar:2.9.4 bin/pulsar standalone
```

Since both VMs are on the same Swedish Science Cloud (NAISS) private network, VM B can connect to the broker on VM A's private IP even with the default port binding.

2.2.4 The Consumer on VM B

On VM B, only the `pulsar-client` library is needed:

```
pip3 install pulsar-client==2.9.4
```

The consumer script is identical in logic to Part A, but the broker URL points to VM A's private IP instead of `localhost`. The implementation of `consumer-task-2-B.py` is shown in Appendix A.

2.2.5 Running the Programs

I started the consumer on VM B first, then ran the producer on VM A. The consumer immediately printed the received message, confirming that the message flowed correctly across the two machines through the Pulsar broker.

```

data-engineering-II (0) ssh, ubuntu
ubuntu@arnab-de2-a2-t1:~$ hostname --ip-address
192.168.2.198 172.17.0.1 fe80::f816:3eff:fe58:eda5 fe80::6088:20ff:fe5d:f8c3 fe80::8c9b:3dff:fe75:7e25
ubuntu@arnab-de2-a2-t1:~$ python3 producer.py
2026-04-20 16:01:22.128 INFO [140402547183296] ExecutorService:41 | Run io_service in a single thread
2026-04-20 16:01:22.128 INFO [140402567090624] ClientConnection:189 | [none] -> pulsar://localhost:6650] Create ClientConnection, timeout=10000
2026-04-20 16:01:22.128 INFO [140402567090624] ConnectionPool:96 | Created connection for pulsar://localhost:6650
2026-04-20 16:01:22.129 INFO [140402547183296] ClientConnection:375 | [127.0.0.1:60414 -> 127.0.0.1:6650] Connected to broker
2026-04-20 16:01:22.135 INFO [140402547183296] HandlerBase:61 | [persistent://public/default/DEtopic, ] Getting connection from pool
2026-04-20 16:01:22.141 INFO [140402547183296] ProducerImpl:173 | [persistent://public/default/DEtopic, ] Created producer on broker [127.0.0.1:60414 -> 127.0.0.1:6650]
2026-04-20 16:01:22.150 INFO [140402567090624] ClientImpl:500 | Closing Pulsar client with 1 producers and 0 consumers
2026-04-20 16:01:22.150 INFO [140402567090624] ProducerImpl:590 | [persistent://public/default/DEtopic, standalone-0-1] Closing producer for topic persistent://public/default/DEtopic
2026-04-20 16:01:22.152 INFO [140402547183296] ProducerImpl:630 | [persistent://public/default/DEtopic, standalone-0-1] Closed producer
2026-04-20 16:01:22.152 INFO [140402530317888] ClientConnection:1561 | [127.0.0.1:60414 -> 127.0.0.1:6650] Connection closed
2026-04-20 16:01:22.152 INFO [140402530317888] ClientConnection:263 | [127.0.0.1:60414 -> 127.0.0.1:6650] Destroyed connection
2026-04-20 16:01:22.153 INFO [140402547183296] ExecutorService:47 | Event loop of ExecutorService exits successfully
2026-04-20 16:01:22.157 INFO [140402567090624] ProducerImpl:559 | Producer = [persistent://public/default/DEtopic, standalone-0-1] , [batching = off]
ubuntu@arnab-de2-a2-t1:~$

data-engineering-II-VM-2 ssh, ubuntu
2026-04-20 16:01:13.526 INFO [140019331983488] ClientConnection:189 | [none] -> pulsar://192.168.2.198:6650] Create ClientConnection, timeout=10000
2026-04-20 16:01:13.526 INFO [140019331983488] ConnectionPool:96 | Created connection for pulsar://192.168.2.198:6650
2026-04-20 16:01:13.528 INFO [140019331916608] ClientConnection:375 | [192.168.2.166:39200 -> 192.168.2.198:6650] Connected to broker
2026-04-20 16:01:13.543 INFO [140019331916608] HandlerBase:61 | [persistent://public/default/DEtopic, DE-sub, 0] Getting connection from pool
2026-04-20 16:01:13.543 INFO [140019222181440] ExecutorService:41 | Run io_service in a single thread
2026-04-20 16:01:13.545 INFO [140019331916608] ClientConnection:189 | [none] -> pulsar://192.168.2.198:6650] Create ClientConnection, timeout=10000
2026-04-20 16:01:13.545 INFO [140019331916608] ConnectionPool:96 | Created connection for pulsar://localhost:6650
2026-04-20 16:01:13.546 INFO [140019331916608] ClientConnection:377 | [192.168.2.166:39204 -> 192.168.2.198:6650] Connected to broker through proxy. Logical broker: pulsar://localhost:6650
2026-04-20 16:01:13.552 INFO [140019331916608] ConsumerImpl:218 | [persistent://public/default/DEtopic, DE-sub, 0] Created consumer on broker [192.168.2.166:39204 -> 192.168.2.198:6650]
Received message : 'b>Welcome to Data Engineering Course!'
2026-04-20 16:01:22.220 INFO [140019331983488] ClientImpl:500 | Closing Pulsar client with 0 producers and 1 consumers
2026-04-20 16:01:22.221 INFO [140019331983488] ConsumerImpl:885 | [persistent://public/default/DEtopic, DE-sub, 0] Closing consumer for topic persistent://public/default/DEtopic
2026-04-20 16:01:22.223 INFO [140019331916608] ConsumerImpl:935 | [persistent://public/default/DEtopic, DE-sub, 0] Closed consumer 0
2026-04-20 16:01:22.223 INFO [140019213788736] ClientConnection:1561 | [192.168.2.166:39200 -> 192.168.2.198:6650] Connection closed
2026-04-20 16:01:22.224 INFO [140019213788736] ClientConnection:263 | [192.168.2.166:39200 -> 192.168.2.198:6650] Destroyed connection
2026-04-20 16:01:22.224 INFO [140019213788736] ClientConnection:1561 | [192.168.2.166:39204 -> 192.168.2.198:6650] Connection closed
2026-04-20 16:01:22.225 INFO [140019213788736] ClientConnection:263 | [192.168.2.166:39204 -> 192.168.2.198:6650] Destroyed connection
2026-04-20 16:01:22.225 INFO [140019331916608] ExecutorService:47 | Event loop of ExecutorService exits successfully
2026-04-20 16:01:22.225 INFO [140019222181440] ExecutorService:47 | Event loop of ExecutorService exits successfully
ubuntu@arnab-de2-a2-t2:~$

```

Figure 3: Consumer on VM B (arnab-de2-a2-t2) receiving a message produced on VM A (arnab-de2-a2-t1), demonstrating cross-machine message delivery via Apache Pulsar.

3 Task 3: Conceptual Questions

1. What are the primary trade-offs when deploying Apache Pulsar in a Docker container versus a native environment?

Using Docker is quick and easy. You can start Pulsar with one command and it works the same on any machine. The downside is that Docker adds a network layer. By default, Pulsar tells clients to connect to its container-internal IP, which other machines cannot reach. You need to set `PULSAR_ADVERTISED_ADDRESS` to fix this. There is also a small performance cost compared to running natively.

Running Pulsar directly on the VM (native) gives better performance and simpler networking. However, you have to install Java, ZooKeeper, BookKeeper, and Pulsar manually, which takes more time and is harder to reproduce on another machine.

2. What features does Apache Pulsar support that previous distributed streaming frameworks, such as Kafka, do not?

Apache Pulsar has several features that Kafka does not have natively:

- **Unified messaging:** Pulsar supports both message queuing and pub-sub in one system. Kafka is mainly a pub-sub log.
- **Multi-tenancy:** Pulsar has built-in support for multiple tenants with separate access control and resource limits. Kafka has no such feature.
- **Geo-replication:** Pulsar can replicate data across data centres out of the box. Kafka needs extra tools like MirrorMaker for this.

- **Tiered storage:** Pulsar can move old data automatically to cheap storage like AWS S3, keeping hot data on fast disks.
- **Pulsar Functions:** a simple built-in way to run stream processing logic without needing a separate engine like Flink or Spark.
- **Decoupled storage:** brokers and storage (BookKeeper) are separate, so you can scale them independently.

3. What is the issue with using a batch processing approach on data at scale? How can Apache Pulsar overcome this?

In batch processing, data is collected for a period of time and then processed all at once. This means results are delayed until the whole batch is done. At large scale, batches become very big, which uses a lot of memory and CPU at once. If something fails, you may need to reprocess the entire batch. You also cannot react to events as they happen.

Apache Pulsar processes each message as soon as it arrives. This gives very low latency. If a message fails, only that message needs to be retried, not the whole dataset. Pulsar stores messages in BookKeeper, so they can be replayed at any time without reprocessing everything from the start.

4. What is the underlying messaging pattern used by Apache Pulsar? What is the advantage of this messaging pattern?

Pulsar uses the **publish-subscribe (pub-sub)** pattern. A producer sends messages to a topic. One or more consumers subscribe to that topic and receive the messages. The producer does not need to know who the consumers are.

The main advantage is that producers and consumers are loosely coupled. You can add or remove consumers without changing the producer. Multiple consumers can also read from the same topic at the same time, which makes the system flexible and easy to scale.

5. What are different modes of subscription? When is each mode used?

Pulsar has four subscription modes:

- **Exclusive:** only one consumer is allowed at a time. Used when you need strict ordering and a single consumer.
- **Shared:** messages are spread across multiple consumers. Used when you want to process messages in parallel and order does not matter.
- **Failover:** one consumer is active and others are on standby. If the active consumer stops, a standby takes over. Used when you need ordering and high availability.
- **Key-shared:** messages with the same key always go to the same consumer. Used when you need per-key ordering but also want parallel processing.

6. What is the role of ZooKeeper? Is it possible to ensure reliability without ZooKeeper?

ZooKeeper keeps track of the cluster state. It stores information about which brokers are running, which broker owns which topic, and cluster configuration. It also helps brokers agree on a leader when a failover happens.

Yes, it is possible to run without ZooKeeper. Pulsar 2.10 and later can use **etcd** or an embedded metadata store instead. Apache Kafka also removed ZooKeeper in version 3.x and replaced

it with its own KRaft protocol. However, you always need some coordination service. Without one, the cluster cannot reliably agree on its state.

7. What are the main architectural components of Apache Pulsar?

- **Brokers:** handle messages from producers and send them to consumers. They do not store data themselves, so they are stateless.
- **BookKeeper (Bookies):** store messages on disk. Multiple bookies replicate the data for durability.
- **ZooKeeper:** stores cluster metadata and coordinates brokers.
- **Producers:** applications that send messages to a topic.
- **Consumers:** applications that read messages from a topic.
- **Pulsar Functions** (optional): run small processing tasks directly inside Pulsar.

8. What is the meaning of broker in Pulsar?

A broker is a server that sits between producers and consumers. It receives messages from producers and delivers them to consumers. It also decides which BookKeeper ledger to write data to. Brokers do not store any data themselves, so if one broker goes down, another can take over its topics without losing any messages.

9. What is the role of a tenant in Pulsar?

A tenant is the highest level of organisation in Pulsar. It represents a team, company, or application that uses the cluster. Each tenant can have its own namespaces, access rules, and resource limits. This way, many different users can share one Pulsar cluster without interfering with each other.

10. What will happen if there is no log compaction in Apache Pulsar?

Without log compaction, the topic log keeps growing as new messages arrive. For topics that track the latest state of something (like a user's settings), old values pile up in the log. A consumer that wants the current state has to read through all the old messages to find it. This wastes storage space and makes replaying the log slow. Log compaction removes old values and keeps only the latest message for each key, so the log stays small and fast to read.

11. What is the difference between Pulsar topics and partitions?

A **topic** is a named stream of messages. Producers write to it and consumers read from it. A **partitioned topic** is a topic that is split into several smaller topics called partitions. Each partition is handled by a different broker. This allows many brokers to work in parallel, which increases throughput. A regular (non-partitioned) topic uses only one broker, which keeps messages strictly ordered but limits how fast data can flow. Within a partition, order is guaranteed; across partitions, it is not.

12. What is the role of acknowledgement in message delivery guarantees?

When a consumer finishes processing a message, it sends an acknowledgement (ack) to the broker. This tells the broker the message was handled successfully and can be removed. If the consumer does not send an ack (because it crashed, for example), the broker will redeliver the message after a timeout. This gives **at-least-once** delivery – every message is guaranteed to

be processed at least once. If a consumer wants the broker to resend a message immediately, it sends a negative acknowledgement (nack). Without acks, the broker has no way to know if a message was processed, so reliable delivery is not possible.

4 Task 4 : Splitting and Merging with Apache Pulsar

4.1 Issue with the Current Implementation

The original `conversion.py` processes each word sequentially in a single loop on one machine. When the number of words grows to millions, this approach becomes a bottleneck because:

- All computation runs on a single thread and a single machine. There is no parallelism.
- Memory usage grows linearly with the number of words since the entire string must be held in memory at once.
- A single failure stops the entire job with no mechanism for retrying just the failed word.
- There is no backpressure: if the conversion is slow, words pile up in memory with no way to buffer or distribute the load.

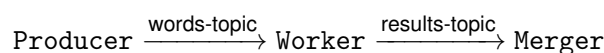
4.2 Redesigned Architecture

The redesigned system replaces the sequential loop with an Apache Pulsar pipeline consisting of three roles:

1. **Producer (Splitter):** reads the input string, splits it into individual words, and publishes each word as a JSON message to the `words-topic`.
2. **Worker consumers / producers:** subscribe to `words-topic` using a *Shared* subscription so multiple workers can consume in parallel. Each worker applies the `conversion()` function to its word and publishes the result to `results-topic`.
3. **Merger consumer:** subscribes to `results-topic`, collects converted words keyed by their original index, and reassembles the final string once all words of a job have arrived.

Each message carries a `job_id`, the word `index`, the word, and `total` (total number of words in the job). This allows the merger to know when a job is complete and reconstruct the original order correctly.

The data flow is:



The *Shared* subscription on `words-topic` means any number of worker processes can be added without changing the producer or merger. The Pulsar broker manages load balancing across workers automatically.

4.3 Implementation

The implementation consists of three Python files:

- `producer-task-4.py`: splits the input string and publishes words to `words-topic`
- `consumer-task-4-worker.py`: applies conversion function and forwards results to `results-topic`
- `consumer-task-4-merger.py`: collects results from `results-topic` and reassembles the final string

The implementation details are shown in Appendix A.

4.4 Running the Programs

All three components run on the same VM, each in a separate terminal session (e.g. with `tmux`). Pulsar must already be running before starting any of the components.

Listing 4: Starting the merger, worker, and producer in separate sessions

```
# Session 1: start the merger first so it is ready to collect results
python3 consumer-task-4-merger.py

# Session 2: start one (or more) worker(s)
python3 consumer-task-4-worker.py

# Session 3: run the producer to split and publish the input string
python3 producer-task-4.py
```

The producer splits the input string into words and publishes them to `words-topic`. The worker picks up each word, converts it to uppercase using `conversion()`, and publishes the result to `results-topic`. The merger collects all converted words and, once it has received all words for the job, prints the final reconstructed string.

The screenshot shows three terminal windows running on a single VM. The top window shows the producer task output, which splits the input string 'I AM WORKING ON ASSIGNMENT 2 FOR DE-II. I WANT TO BE CAPATILIZED' into words and publishes them. The middle window shows the worker task output, which receives each word, converts it to uppercase (e.g., 'I' to 'I'), and publishes the result. The bottom window shows the merger task output, which receives all converted words and prints the final reconstructed string: '2 FOR DE-II. I WANT TO BE CAPATILIZED'.

Figure 4: Task 4 running on a single VM: the producer splits the input string, the worker applies the `conversion()` function to each word, and the merger reassembles the converted string.

A Code Files

A.1 Task 2.A: Consumer

Listing 5: consumer-task-2-A.py

```
1 """
2 Task 2: Consumer
3
4 Create a Pulsar consumer that receives a message from a topic.
5 """
6
7 import pulsar
8
9
10 def main():
11     # Create a pulsar client by supplying ip address and port
12     client = pulsar.Client('pulsar://localhost:6650')
13     # Subscribe to a topic and subscription
14     consumer = client.subscribe('DEtopic', subscription_name='DE-sub')
15     # Display message received from producer
16     msg = consumer.receive()
17     try:
18         print("Received message : '%s'" % msg.data())
19         # Acknowledge for receiving the message
20         consumer.acknowledge(msg)
21     except:
22         consumer.negative_acknowledge(msg)
23     # Destroy pulsar client
24     client.close()
25
26
27 if __name__ == "__main__":
28     main()
```

A.2 Task 2.A: Producer

Listing 6: producer-task-2.py

```
1 """
2 Task 2: Producer
3
4 Create a Pulsar producer that sends a message to a topic.
5 """
6
7 import pulsar
8
9
10 def main():
11     # Create a pulsar client by supplying ip address and port
12     client = pulsar.Client('pulsar://localhost:6650')
13     # Create a producer on the topic that consumer can subscribe to
14     producer = client.create_producer('DEtopic')
15     # Send a message to consumer
16     producer.send(('Welcome to Data Engineering Course!').encode('utf-8'))
17     # Destroy pulsar client
18     client.close()
19
20
```

```

21 if __name__ == "__main__":
22     main()

```

A.3 Task 2.B: Consumer

Listing 7: consumer-task-2-B.py

```

1  """
2  Task 2: Consumer
3
4  Create a Pulsar consumer that receives a message from a topic.
5  """
6
7  import pulsar
8
9
10 def main():
11     # Create a pulsar client by supplying ip address and port
12     client = pulsar.Client('pulsar://192.168.2.198:6650')
13     # Subscribe to a topic and subscription
14     consumer = client.subscribe('DEtopic', subscription_name='DE-sub')
15     # Display message received from producer
16     msg = consumer.receive()
17     try:
18         print("Received message : '%s'" % msg.data())
19         # Acknowledge for receiving the message
20         consumer.acknowledge(msg)
21     except:
22         consumer.negative_acknowledge(msg)
23     # Destroy pulsar client
24     client.close()
25
26
27 if __name__ == "__main__":
28     main()

```

A.4 Task 4: Producer

Listing 8: producer-task-4.py

```

1  """
2  Task 4: Producer (Splitter)
3
4  Splits the INPUT_STRING into individual words and publishes each word as a separate
5  Pulsar message to the 'words-topic' topic.
6  Every message contains a JSON payload:
7  - job_id : unique identifier for this processing job
8  - index  : position of the word in the original string
9  - word   : the individual word to be processed
10 - total  : total number of words in the original string (for merger to know when
11           all words have arrived)
12 """
13
14 import json
15 import uuid
16 import pulsar
17
18 INPUT_STRING = "I am working on assignment 2 for DE-II. I want to be capatilized"

```

```

17
18 def main():
19     words    = INPUT_STRING.split()
20     job_id   = str(uuid.uuid4())
21
22     client    = pulsar.Client('pulsar://localhost:6650')
23     producer  = client.create_producer("words-topic")
24
25     print(f"Producer Job: {job_id}")
26     print(f"Producer Splitting '{INPUT_STRING}' into {len(words)} words ...")
27
28     for index, word in enumerate(words):
29         payload = json.dumps({
30             "job_id": job_id,
31             "index":  index,
32             "word":   word,
33             "total":  len(words),
34         }).encode("utf-8")
35
36         producer.send(payload)
37         print(f"Producer sent word[{index}] = '{word}'")
38
39     print("All words published.")
40     client.close()
41
42
43 if __name__ == "__main__":
44     main()

```

A.5 Task 4: Worker

Listing 9: consumer-task-4-worker.py

```

1  """
2  Task 4: Worker Consumer / Producer
3
4  Subscribes to the 'words-topic'. For each incoming word message it:
5      1. Applies the conversion() function from conversion.py.
6      2. Publishes the converted word to 'results-topic'.
7  """
8
9  import json
10 import pulsar
11
12
13 def function(string):
14     return string.upper()
15
16
17 def conversion(substring, operation):
18     return operation(substring)
19
20
21 def main():
22     client    = pulsar.Client('pulsar://localhost:6650')
23     consumer  = client.subscribe(
24         'words-topic',
25         subscription_name='workers-sub',

```

```

26     consumer_type=pulsar.ConsumerType.Shared,
27 )
28 producer = client.create_producer('results-topic')
29
30 print("Worker waiting for words...")
31
32 while True:
33     msg = consumer.receive()
34     try:
35         data = json.loads(msg.data().decode("utf-8"))
36         word = data["word"]
37         converted = conversion(word, function)
38
39         result = json.dumps({
40             "job_id": data["job_id"],
41             "index": data["index"],
42             "word": converted,
43             "total": data["total"],
44         }).encode("utf-8")
45
46         producer.send(result)
47         print(f"Worker converted word[{data['index']}]: '{word}' ->
48 '{converted}'")
49         consumer.acknowledge(msg)
50     except Exception as e:
51         print(f"Worker error: {e}")
52         consumer.negative_acknowledge(msg)
53
54 if __name__ == "__main__":
55     main()

```

A.6 Task 4: Merger

Listing 10: consumer-task-4-merger.py

```

1  """
2  Task 4: Merger Consumer
3
4  Subscribes to the 'results-topic'. Collects converted words with original index.
5  Once all words for a job have arrived it reassembles them in order and prints the
6  final converted string.
7  """
8
9  import json
10 import pulsar
11
12 def main():
13     client = pulsar.Client('pulsar://localhost:6650')
14     consumer = client.subscribe(
15         'results-topic',
16         subscription_name='merger-sub',
17     )
18
19     print("Merger waiting for converted words...")
20
21     jobs = {} # job_id -> {"words": {index: word}, "total": int}

```

```

22
23 while True:
24     msg = consumer.receive()
25     try:
26         data = json.loads(msg.data().decode("utf-8"))
27         job_id = data["job_id"]
28         index = data["index"]
29         word = data["word"]
30         total = data["total"]
31
32         print(f"Merger received word[{index}] = '{word}' (job: {job_id})")
33
34         if job_id not in jobs:
35             jobs[job_id] = {"words": {}, "total": total}
36             jobs[job_id]["words"][index] = word
37
38         if len(jobs[job_id]["words"]) == jobs[job_id]["total"]:
39             merged = " ".join(jobs[job_id]["words"][i] for i in range(total))
40             print(f"Merged result (job: {job_id}): '{merged}'")
41             del jobs[job_id]
42
43         consumer.acknowledge(msg)
44
45     except Exception as e:
46         print(f"Merger error: {e}")
47         consumer.negative_acknowledge(msg)
48
49
50 if __name__ == "__main__":
51     main()

```

B VM Setup Script

I used the following script to provision the VM with all required dependencies before running Pulsar and the Python programs.

Listing 11: setup_vm.sh

```

#!/bin/bash
# Setup script for Data Engineering II - Assignment 2
# Prerequisites: Ubuntu 22.04+, Python3 (preinstalled)
# Run as: bash setup_vm.sh

set -e

echo "====="
echo " DE2 A2 - VM Setup Script"
echo "====="

# System update
echo "[1/4] Updating system packages..."
sudo apt update -y && sudo apt upgrade -y

# Java Runtime Environment
echo "[2/4] Installing Java Runtime Environment..."
sudo apt install -y default-jre
java -version

```

```

# Set JAVA_HOME
JAVA_BIN=$(which java)
JAVA_RESOLVED=$(readlink -f "$JAVA_BIN")
JAVA_HOME_PATH=$(dirname "$(dirname "$JAVA_RESOLVED")")
echo "export JAVA_HOME=${JAVA_HOME_PATH}" | sudo tee -a /etc/environment
echo "export JAVA_HOME=${JAVA_HOME_PATH}" >> ~/.bashrc
export JAVA_HOME=${JAVA_HOME_PATH}
echo "JAVA_HOME set to: $JAVA_HOME"

# Docker
echo "[3/4] Installing Docker..."

# Remove old/conflicting versions if any
sudo apt remove -y $(dpkg --get-selections docker.io docker-compose docker-compose-v2
    docker-doc podman-docker containerd runc 2>/dev/null | cut -f1) 2>/dev/null ||
    true

sudo apt install -y ca-certificates curl

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
    docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add Docker apt repository (DEB822 format)
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update -y
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
    compose-plugin

# Ensure Docker service is running
sudo systemctl start docker
sudo systemctl enable docker

# Allow current user to run docker without sudo
sudo usermod -aG docker "$USER"

docker --version

# Python pulsar-client
echo "[4/4] Installing pulsar-client..."
sudo apt install -y python3-pip
pip3 install pulsar-client==2.9.4

echo ""
echo "====="
echo " Setup Complete!"
echo "====="
echo ""

```